

UNIVERSITAT POLITÈCNICA DE CATALUNYA

GRAU EN INTEL·LENCIA ARTIFICIAL

BASES DE DADES AVANÇADES



Project 12
Semantic Data Management with Knowledge
Graphs

Pablo Avilés, Joel Blanco, Marc Landa, Marina Teruel

29/05/2026

Table of Contents

1. Introduction.....	3
2. Datasets.....	3
3. Architecture.....	4
4. Knowledge Graph model.....	4
5. SPARQL analysis pipeline.....	5
6. Knowledge Graph Embeddings pipeline.....	6
7. Limitations and conclusion.....	7

1. Introduction

This second project extends the DataOps pipeline developed in Project 1, but changes the main analytical goal. In Project 1, the final exploitation layer was tabular and the analysis was based on classical machine learning tasks, mainly regression and clustering over daily accident, weather and Uber indicators. In Project 2, we reuse the same data engineering structure, but the exploitation layer becomes semantic: the integrated data is represented as a Knowledge Graph in RDF/RDFS, where days, weather observations, mobility observations, borough-level safety records, vehicle types and contributing factors are explicitly connected.

The goal is therefore not to repeat the previous tabular analysis, but to show that semantic modelling adds value. Once the Knowledge Graph is built, it is exploited in two complementary ways: first, through SPARQL queries that perform interpretable pattern matching over graph paths; and second, through Knowledge Graph Embeddings, which are used for a link prediction task that ranks plausible contributing factors for each day. In this way, the project moves from flat feature-based analysis to graph-oriented semantic exploitation.

2. Datasets

Uber NYC 2016. The Uber dataset contains 1,474 rows and 10 columns after ingestion. It describes Uber activity in New York City during 2016, including variables such as base license number, base name, pickup start and end dates, total dispatched trips and unique dispatched vehicles. Its original granularity is weekly and city/base-level, so it provides a general proxy for mobility demand rather than detailed borough-level movement.

Weather NYC 2016. The weather dataset contains 366 rows and 7 columns after ingestion. It provides daily meteorological information for New York City, including maximum, minimum and average temperature, precipitation, snowfall and snow depth. Its granularity is daily and city-level, so each record represents the general weather conditions of the city for a specific day.

NYC Vehicle Collisions. The collisions dataset contains 477,732 rows and 29 columns after ingestion. Each row represents an individual police-reported traffic collision, with information such as date, time, borough, location, number of injured or killed people, vehicle types and contributing factors. Its granularity is event-level, which makes it the most detailed dataset and the richest one for semantic modelling.

Since the datasets have different granularities, they are integrated through the only common and reliable dimension: the date. This allows weekly Uber activity, daily weather conditions and individual collision records to be aligned at a shared daily level without forcing unsupported spatial joins.

3. Architecture

As in Project 1, this project follows a zone-based architecture, organizing the data engineering pipeline into four main stages:

- **Landing Zone:** in this first phase, we import the three raw datasets from Kaggle. Since they are originally provided in CSV format, they are converted into Parquet to improve storage efficiency and downstream processing.
- **Formatted Zone:** the datasets are standardized into relational tables using Spark and DuckDB. This stage mainly focuses on syntactic homogenization, such as column naming, type conversion and table creation.
- **Trusted Zone:** in this stage, we apply denial constraints and additional quality checks to validate and clean the datasets. The cleaned results are stored again in DuckDB and become the trusted input for the semantic layer.
- **KG-based Exploitation Zone:** instead of producing only one final tabular dataset as in Project 1, this phase builds a Knowledge Graph from the trusted data. The graph is materialized in `exploitation_zone.db` through node, edge and literal tables, and it is also exported as `exploitation_zone.ttl` in RDF/RDFS Turtle format.

After this, the graph is exploited by two analysis pipelines: a SPARQL pipeline for semantic pattern matching and a Knowledge Graph Embeddings pipeline for factor recommendation. The execution is organized through Python scripts, with `run_pipeline.py` acting as the project-level runner that launches the selected stages and produces the final databases, graph files and analysis outputs.

4. Knowledge Graph model

The Knowledge Graph is organized around Day, which acts as the common semantic hub for the three datasets. The main structure is summarized below:

Source/context	KG path	Main literals
Weather	Day → WeatherObservation	averageTemperature, precipitation, snowFall, snowDepth
Uber mobility	Day → MobilityObservation	estimatedDispatchedTrips, estimatedUniqueVehicles, activeBases
Safety by borough	Day → SafetyObservation → Borough	collisions, personsInjured, personsKilled
Contributing factors	Day → FactorCount → ContributingFactor	occurrences
Vehicle types	Day → VehicleTypeCount → VehicleType	occurrences

This structure reflects the different roles of the datasets. Weather and Uber provide city-level daily context, while collision data adds borough-level safety information and richer semantic dimensions such as contributing factors and vehicle types. FactorCount and VehicleTypeCount are modelled as intermediate nodes because the connection between a day and a factor or vehicle type includes an additional value: the number of occurrences.

5. SPARQL analysis pipeline

The SPARQL pipeline is the explicit graph-querying part of the project. Before defining the analysis queries, we first created a small smoke-test query to verify that the KG was correctly loaded and that the main paths from **Day** to weather, mobility and safety observations were working. The detailed objective and graph path of each query are documented in the `.sparql` files and in the query catalogue, so this section focuses on the main results obtained.

Instead of querying a single flat analytical table, the queries exploit the structure of the Knowledge Graph by traversing paths from the central **Day** node to weather observations, mobility observations, borough-level safety observations, contributing factors and vehicle types. This is especially useful because the original datasets have different granularities. Weather and Uber activity are city-level sources, while collisions contain boroughs, vehicle types and contributing factors. SPARQL allows these elements to be combined through shared graph paths without assuming that all variables originally existed at the same spatial resolution.

01_high_collision_days_contributing_factors.sparql - The first analysis, based on high-collision borough-days, shows that Brooklyn appears most often in the highest collision contexts. The strongest contributing factor is **DRIVER INATTENTION/DISTRACTION**, especially for Brooklyn, Queens and Manhattan. Other relevant factors are **FAILURE TO YIELD RIGHT-OF-WAY**, **FATIGUED/DROWSY**, **BACKING UNSAFELY** and **FOLLOWING TOO CLOSELY**. This result is useful because it connects borough-level safety observations with contributing-factor information through the shared day.

02_adverse_weather_vehicle_types.sparql - The adverse-weather query connects weather observations with vehicle-type counts. The results show that **PASSENGER VEHICLE** is by far the most frequent vehicle type on days with precipitation or snow, followed by **SPORT UTILITY/STATION WAGON** and **TAXI**. Although this is expected because passenger vehicles are common overall, the query shows how the KG can relate weather context with vehicle involvement without building a specific flat feature table for that purpose.

03_weekend_vs_weekday_factor_profile.sparql - The weekend-weekday comparison shows different factor profiles depending on the type of day. Frequent factors such as **DRIVER INATTENTION/DISTRACTION** and **FAILURE TO YIELD RIGHT-OF-WAY** have higher average daily occurrence on weekdays, while **ALCOHOL INVOLVEMENT** is relatively more important on weekends. This gives an interpretable example of how calendar information stored in **Day** nodes can be combined with accident semantics.

04_high_mobility_borough_collision_risk.sparql - The high-mobility query studies borough collision levels on days with above-average Uber activity. Since Uber data is only available at city level, the query does not assign trips to boroughs. Instead, it uses the day as the connection point between city-level mobility and borough-level safety observations. The results show that Brooklyn has the highest average collisions on high-mobility days, followed by Queens and Manhattan.

05_weather_factor_vehicle_paths.sparql - Finally, the weather-factor-vehicle path query is mainly explanatory. It returns concrete examples where the same day connects adverse weather, contributing factors and vehicle types. For instance, the rainy/snowy day **2016-01-23** appears with **PAVEMENT SLIPPERY** and vehicle types such as **PASSENGER VEHICLE** and **SPORT UTILITY/STATION WAGON**. This does not mean that all these elements belong to the same individual accident, but it illustrates how the KG connects reusable semantic entities through meaningful relations.

6. Knowledge Graph Embeddings pipeline

The second analysis pipeline uses Knowledge Graph Embeddings for a link prediction task. Instead of predicting the number of accidents, the objective is to rank the most plausible contributing factors for a given day:

(Day, hasLikelyContributingFactor, ContributingFactor)

This target relation is derived from the existing graph path $\text{Day} \rightarrow \text{FactorCount} \rightarrow \text{ContributingFactor}$. However, to avoid leakage, the original `hasFactorCount` and `aboutFactor` relations are excluded from the training context. Otherwise, the model could recover the target relation almost directly from the original graph structure. The model therefore learns from the remaining graph context: weather, mobility, borough-level safety observations, vehicle types and derived categorical context nodes.

Since KGE models operate over triples, part of the numerical context is transformed into categorical graph nodes, such as temperature level, precipitation level, snow condition, mobility level and borough collision level. This keeps the task graph-based while allowing the model to use weather, mobility and safety context. We also keep only factor links with at least 5 daily occurrences to reduce noise from very rare factors. During the split, at least one training example is preserved for each day and each contributing factor, so validation and test triples do not contain completely unseen entities.

The pipeline uses 18,174 context triples and 3,685 target triples. The target triples are divided into 2,580 training triples, 368 validation triples and 737 test triples. We compare several KGE models: TransE, DistMult, ComplEx and RotatE. A small grid search is also applied to TransE, testing different embedding dimensions and learning rates, so the final model is not selected only by intuition.

Model/run	MRR	Hits@1	Hits@3	Hits@10
RotatE_dim64_lr0.01_ep25_neg16	0.6514	0.5292	0.7408	0.8521

TransE_dim32_lr0.01_ep25_neg16	0.4360	0.3094	0.4925	0.6934
TransE_dim64_lr0.01_ep25_neg16	0.4292	0.2972	0.4796	0.7198
TransE_dim64_lr0.005_ep25_neg16	0.3891	0.2626	0.4288	0.6560
DistMult_dim64_lr0.01_ep25_neg16	0.3131	0.2361	0.3263	0.4905
ComplEx_dim64_lr0.01_ep25_neg16	0.0792	0.0197	0.0726	0.1825

The best model is RotatE_dim64_lr0.01_ep25_neg16. Its Hits@10 of 0.8521 means that, in approximately 85% of test cases, the correct contributing factor appears within the top 10 ranked candidates. Hits@3 is also high, with 0.7408, meaning that the correct factor often appears among the first three recommendations.

The final output is a list of factor recommendations for selected days. Each row includes the day, the recommended factor, the model score, whether the factor was observed in the full KG, and contextual information such as average temperature, precipitation, estimated Uber trips, total collisions and top borough. The score is not interpreted as a probability, but as a ranking value used to order plausible factors.

These recommendations should not be understood as causal explanations or direct accident prevention rules. They indicate factors that the model considers plausible according to the graph structure and the patterns learned from the KG.

7. Limitations and conclusion

The main limitations come from the scope and granularity of the data. The project only uses 2016 data, so temporal generalization is not evaluated. Also, Uber and weather are city-level sources, while collisions include borough-level information, so borough-specific conclusions mainly come from the accident dataset. Finally, the KGE recommendations should be understood as plausible contributing factors learned from the graph, not as causal explanations or direct accident prevention rules.

Overall, Project 2 keeps the DataOps foundation from Project 1 but changes the exploitation paradigm. Instead of producing only a flat analytical table, the exploitation zone now produces a Knowledge Graph in RDF/RDFS. This graph makes the semantics of the data explicit and allows the project to exploit relationships between days, weather, mobility, boroughs, vehicle types and contributing factors.

The two analysis pipelines show this semantic shift from different perspectives. SPARQL provides explicit and interpretable pattern matching over graph paths, while Knowledge Graph Embeddings learn latent representations of entities and relations for link prediction and factor recommendation. Therefore, the main contribution of this project is the transition from tabular exploitation to semantic exploitation, showing how the same data engineering pipeline can be extended with Knowledge Graph technologies to support more expressive and graph-oriented analysis.